Lab 7: Fuzzing for Input Validation Bugs

Name	SRN	Section	Assigned repository
Dhanush S S	PES1UG24AM082	B	L6_L7_19_parking_fee (Lab 7)

Application: `parking_fee.py` has three functions: `minutes_parked` (parses "HH:MM-HH:MM" and returns minutes parked), `hourly_rate` (returns the hourly rate for bike/car/truck) and `format_plate` (formats "ka01ab1234" as "KA 01 AB 1234"). **Tools:** Python 3, pytest 9.2.0, Hypothesis 6.170.15.

Step 1: Install dependencies

```
pip install hypothesis
pip install pytest
```

Step 2: Run the buggy application

Command: `python parking_fee.py`. With well-formed input the original program works:

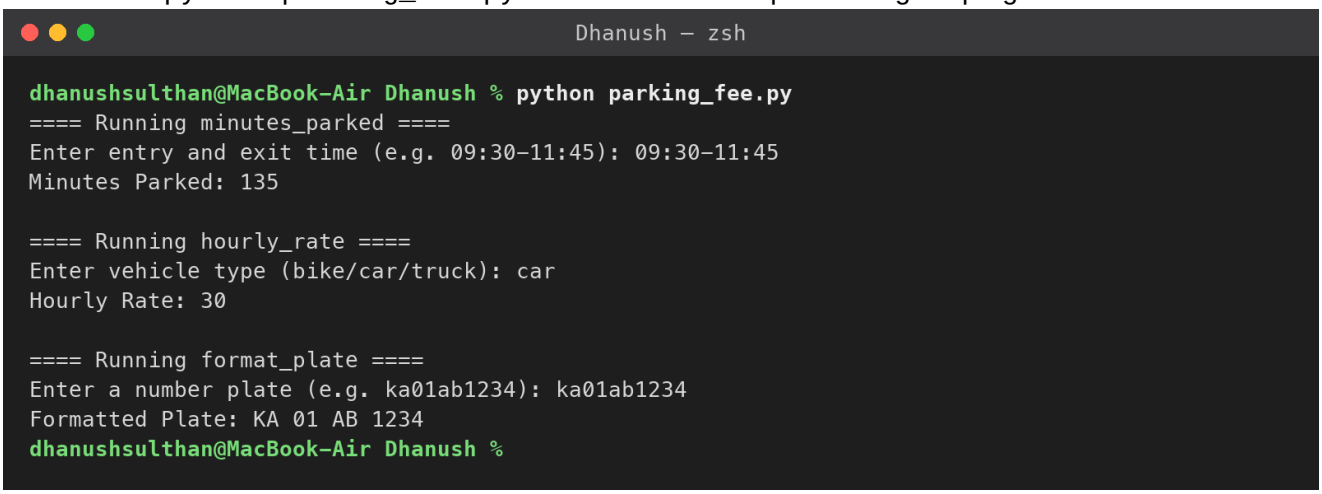


Figure 1: Buggy program with valid input

With unexpected input it crashes or gives wrong answers. A free-text time crashes with **ValueError**. Exit before entry gives **-45 minutes**. An unknown vehicle crashes with **KeyError**. A short plate prints a broken value (`'KA 01 '`) or crashes with **IndexError**.

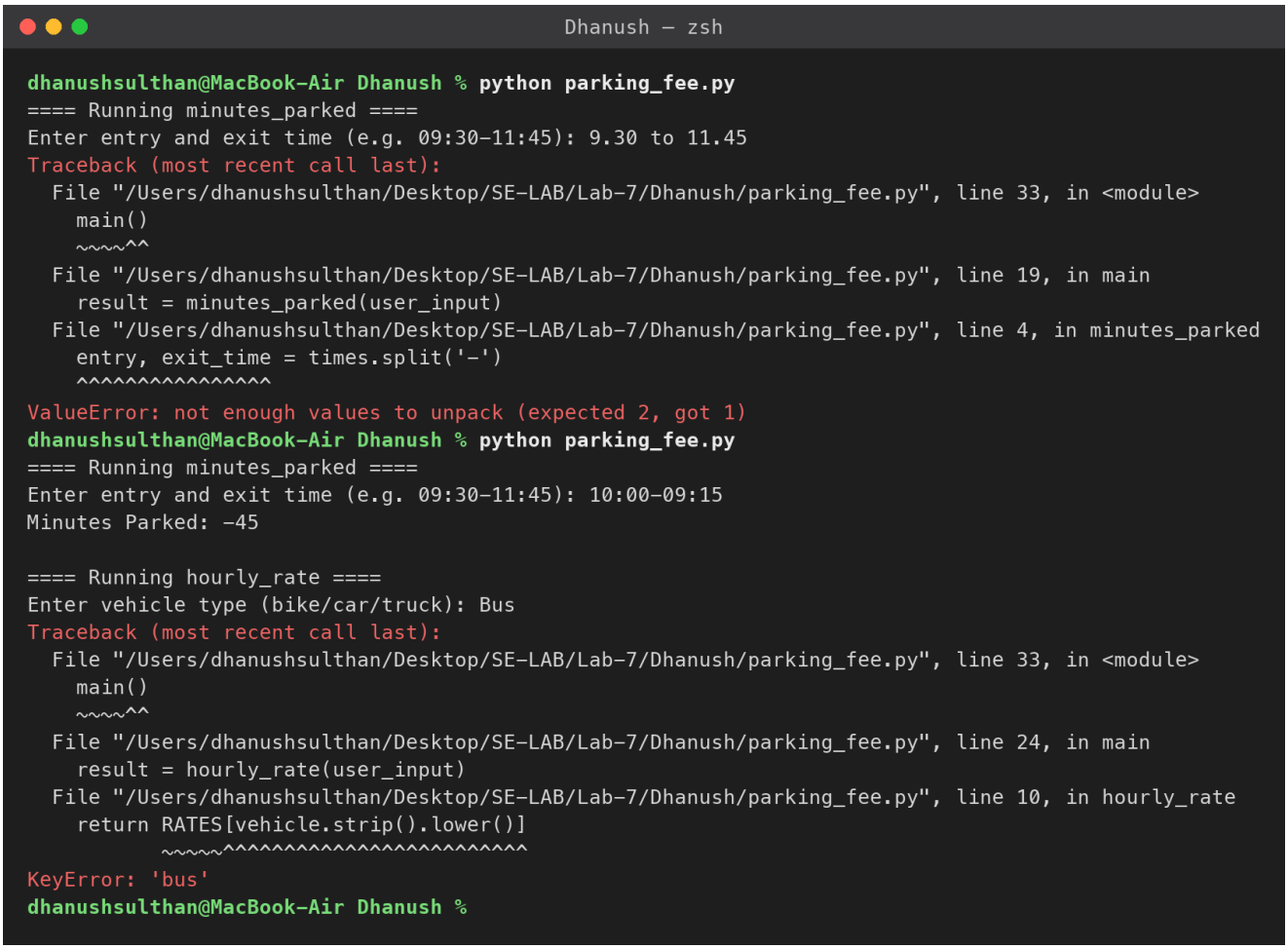


Figure 2: Buggy program - ValueError, negative duration, KeyError

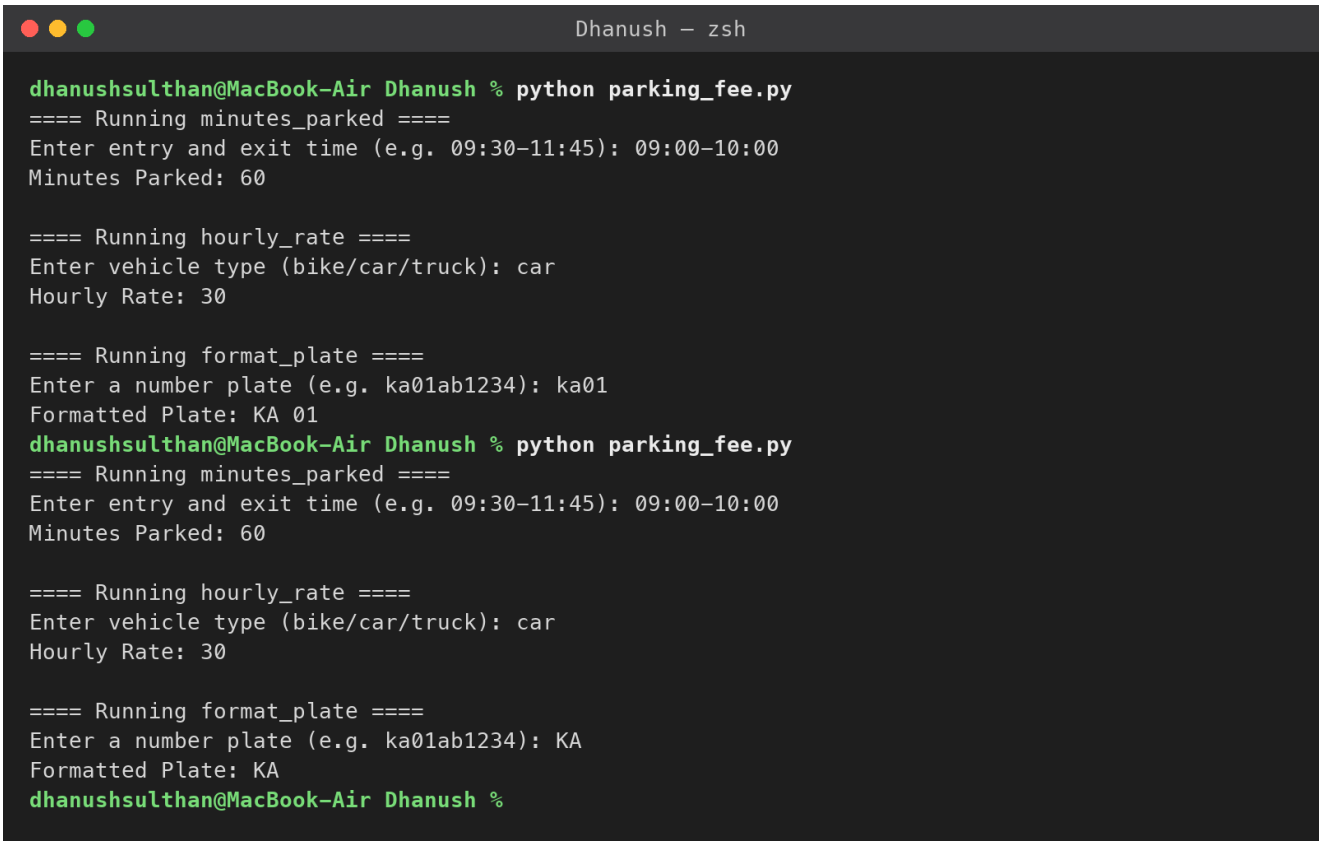


Figure 3: Buggy program - malformed plate output and IndexError

Step 3: Complete test_parking_fee.py using Hypothesis

The starter file had three empty `@given(st.text() | st.none())` tests. I completed them as "no crash" tests: any exception means the test fails. I also added property tests to find wrong results as well as crashes. 10 tests in total:

Test	Strategy (inputs)	Property checked
test_minutes_parked_no_crash	st.text() | st.none()	never raises
test_minutes_parked_no_crash_on_shaped_input	text, None, ints, floats, lists + regex-shaped 'HH:MM-HH:MM' strings	never raises
test_minutes_parked_never_negative_or_too_long	same as above	result is None or 0 <= result <= 1440
test_minutes_parked_valid_times	random valid hh, mm pairs	correct minutes; None when exit is before entry
test_hourly_rate_no_crash	st.text() | st.none()	never raises
test_hourly_rate_returns_known_rate_or_none	text, None, ints, floats, lists	result in {None, 10, 30, 60}
test_hourly_rate_valid_vehicles	bike/car/truck, random case and whitespace	correct rate
test_format_plate_no_crash	st.text() | st.none()	never raises
test_format_plate_output_is_well_formed	any input + plate-like strings	None or exactly 'AA 00 AA 0000'
test_format_plate_valid_plates	regex-generated valid plates (200 examples)	correct formatting

Example (no-crash test):

```
@given(st.text() | st.none())
def test_minutes_parked_no_crash(s):
    try:
        minutes_parked(s)
    except Exception as e:
        assert False, f"minutes_parked crashed with: {e!r}"
```

Step 4: Run test_parking_fee.py on the buggy code - tests failing

Command: `python -m pytest test_parking_fee.py`. **8 of 10 tests fail.** Only the two "valid input" tests pass.

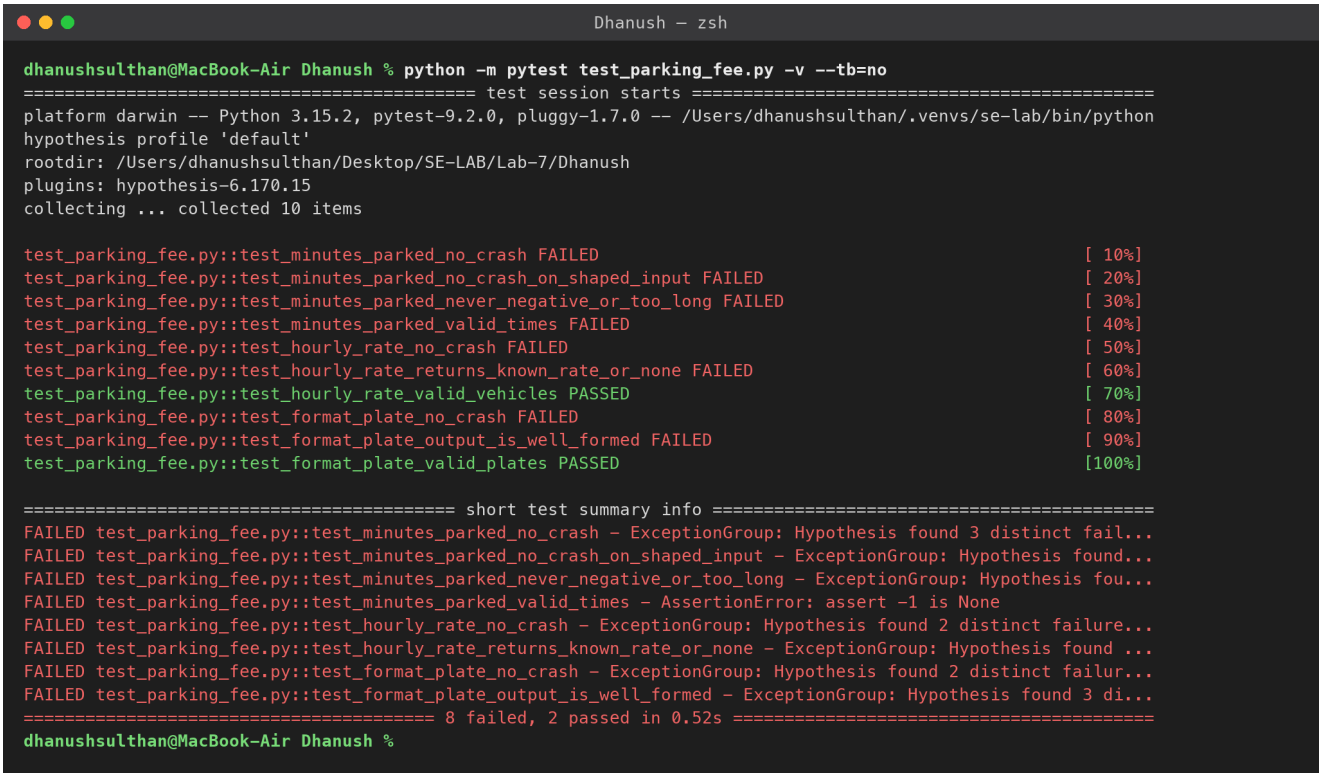


Figure 4: 8 failed, 2 passed on the buggy code

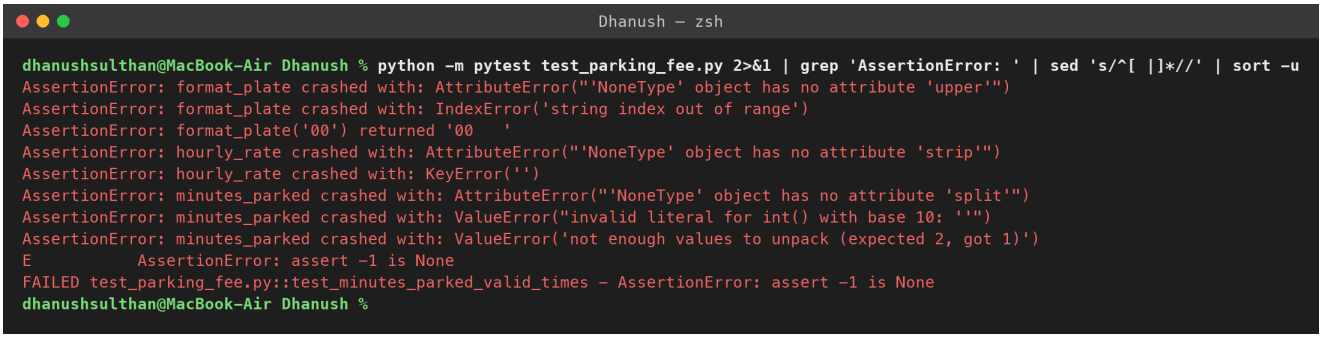


Figure 5: Distinct errors Hypothesis found



Figure 6: Minimal failing examples shrunk by Hypothesis

Bugs discovered

Bugs 1-4 and 6-10 were found by the Hypothesis tests. Bug 5 was found by code review while writing the fix.

#	Function	Minimal failing input	Buggy behaviour	Fix
1	minutes_parked	None	AttributeError: no attribute 'split'	Return None for non-strings
2	minutes_parked	'' or '-'	ValueError: not enough values to unpack	Validate whole string with regex HH:MM-HH:MM
3	minutes_parked	':-:'	ValueError: invalid literal for int()	Regex only accepts ASCII digits [0-9]
4	minutes_parked	'00:01-00:00'	Returns -1 (negative duration)	Return None when exit is before entry
5	minutes_parked	'25:00-26:00'	Accepts impossible times	Check 0-23 hours and 0-59 minutes
6	hourly_rate	None	AttributeError: no attribute 'strip'	Return None for non-strings
7	hourly_rate	'' / 'Bus'	KeyError	Use RATES.get() so unknown vehicles give None
8	format_plate	None	AttributeError: no attribute 'upper'	Return None for non-strings
9	format_plate	''	IndexError: string index out of range	Validate format before indexing
10	format_plate	'00'	Returns malformed '00 '	Require AA00AA0000 (ASCII only, spaces/hyphens ignored)

Approach: every function now validates its input type and format before using it. It returns `None` for input it cannot handle instead of raising or returning a wrong value. `main()` checks for None and prints a helpful message instead of a traceback. Non-ASCII plates are rejected before `.upper()`, because some Unicode characters change length or turn into ASCII letters when upper-cased.

Step 5: Fixed application

```
"""Lab 7 - Fixed parking_fee.py (hardened against invalid input).

Name: Dhanush S S | SRN: PES1UG24AM082 | Section: B

Every function now returns None for input it cannot handle, instead of
crashing or returning a wrong value.
"""
import re

RATES = {"bike": 10, "car": 30, "truck": 60}

# HH:MM-HH:MM with ASCII digits only, spaces allowed around the parts.
TIME_RANGE = re.compile(r"\s*([0-9]{1,2}):([0-9]{2})\s*-\s*([0-9]{1,2}):([0-9]{2})\s*")

# Indian-style plate: 2 letters, 2 digits, 2 letters, 4 digits (e.g. KA01AB1234).
PLATE = re.compile(r"[A-Z]{2}[0-9]{2}[A-Z]{2}[0-9]{4}")


def _to_minutes(hours, mins):
    """Convert 'HH', 'MM' strings to minutes since midnight, or None."""
    h, m = int(hours), int(mins)
    if not (0 <= h <= 23 and 0 <= m <= 59):
        return None
    return h * 60 + m
def minutes_parked(times):
```

```
    """Minutes between entry and exit in 'HH:MM-HH:MM', or None if invalid."""
    if not isinstance(times, str):
        return None
    match = TIME_RANGE.fullmatch(times)
    if match is None:
        return None
    entry = _to_minutes(match.group(1), match.group(2))
    exit_time = _to_minutes(match.group(3), match.group(4))
    if entry is None or exit_time is None or exit_time < entry:
        return None
    return exit_time - entry


def hourly_rate(vehicle):
    """Hourly rate for bike/car/truck (any case, extra spaces OK), or None."""
    if not isinstance(vehicle, str):
        return None
    return RATES.get(vehicle.strip().lower())


def format_plate(plate):
    """Format 'ka01ab1234' as 'KA 01 AB 1234', or None if not a valid plate."""
    if not isinstance(plate, str) or not plate.isascii():
        return None
    p = plate.replace(" ", "").replace("-", "").upper()
    if not PLATE.fullmatch(p):
        return None
    return p[0:2] + " " + p[2:4] + " " + p[4:6] + " " + p[6:10]
```

Command: `python parking_fee.py` (fixed version)

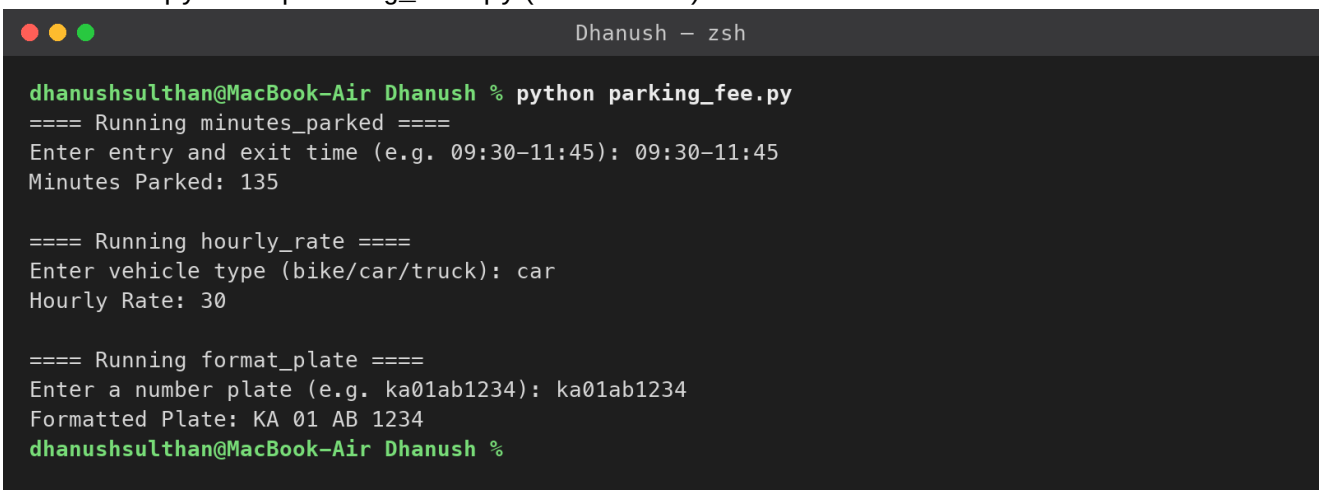


Figure 7: Fixed program with valid input - same results as before

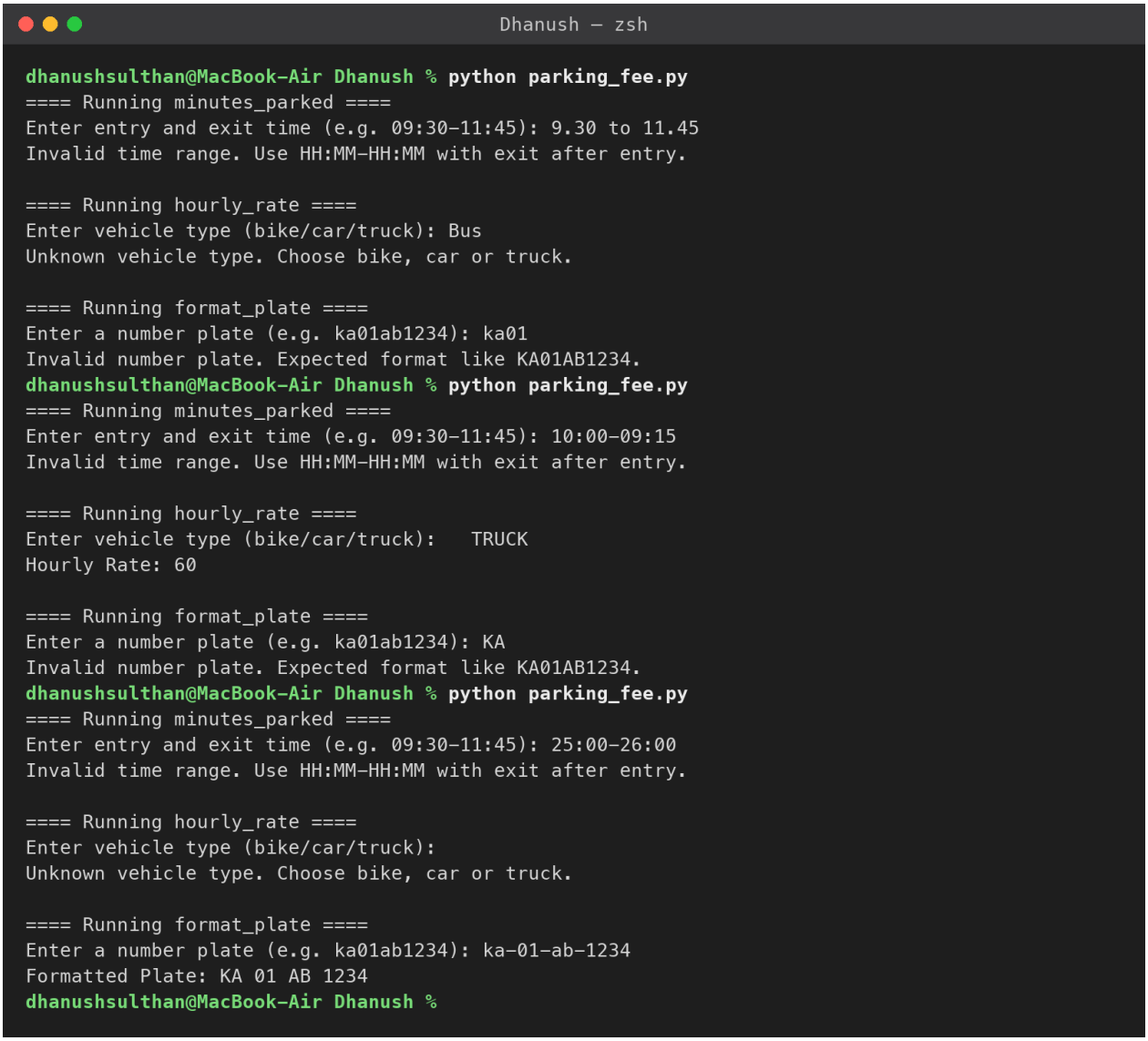


Figure 8: Fixed program with the inputs that crashed it before - no crashes, clear messages

Step 6: Re-run test_parking_fee.py on the fixed code

Command: `python -m pytest test_parking_fee.py -v`. **All 10 tests pass.** I also ran the suite with 8 different `--hypothesis-seed` values and every run passed.

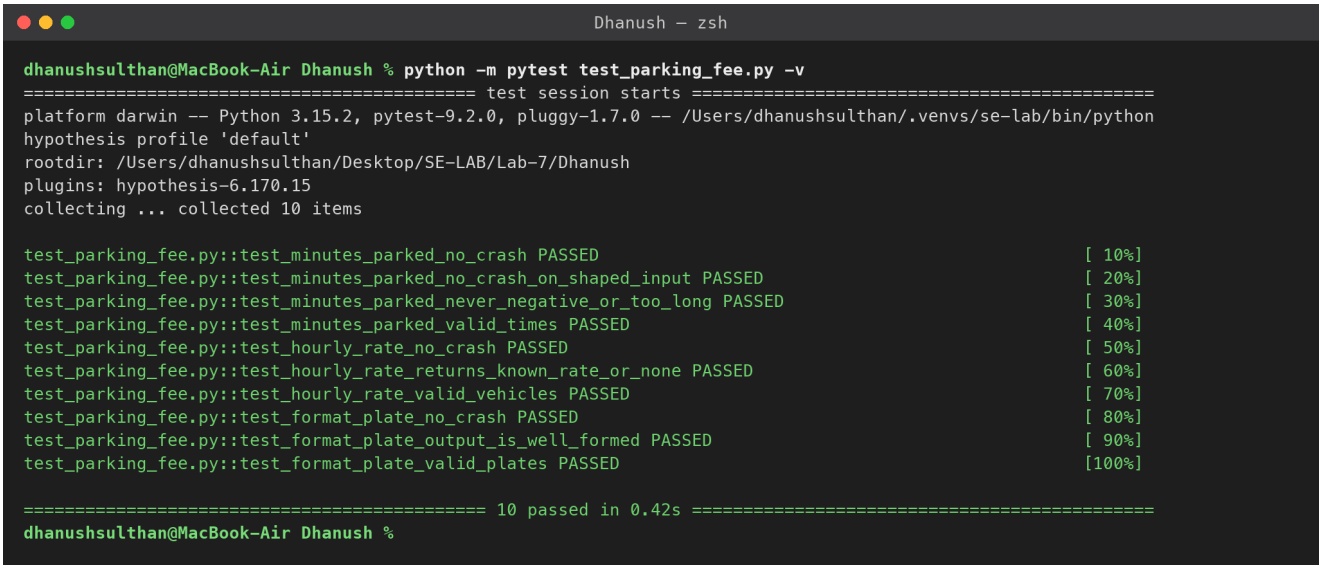


Figure 9: 10 passed on the fixed code

Conclusion

The hand-written examples from the lab (valid times, known vehicles, full plates) all worked on the original code. Hypothesis found 9 input-validation bugs in under a second. It also shrank each one to a minimal input such as `''`, `None`, `'00'` or `'00:01-00:00'`. After adding type checks, format validation and range checks, the same fuzz tests pass, and valid input still gives the same results as before.